HTTP di Golang

Pemrograman Web • 14 slides • Generated on 11 October 2026, 22:42

Dasar HTTP pada Golang

Dasar HTTP di Go

Slide 1

- **Materi:**
  - Package `net/http`
  - Anatomi handler: `func(w http.ResponseWriter, r *http.Request)`
  - `ServeMux` — router bawaan Go
  - Routing berdasarkan method dan path parameter
  - Status code, header, body

Web Server di Go

Slide 2

- Web server = program yang **listen** di port, menunggu request, mengirim response
- Di bahasa lain, program kita **bukan** servernya:
  - PHP → Apache/Nginx yang listen, PHP-FPM yang eksekusi
  - Node.js → Express di atas runtime
  - Python → Flask/Django di belakang Gunicorn
- **Di Go: program kita sendiri yang jadi server**
  - Tidak ada Apache/Nginx di depannya
  - HTTP server sudah ada di standard library
  - `go build` → satu binary, web server di dalamnya
- Konsekuensinya:

- `go run main.go` → port langsung terbuka
- Ctrl+C → server mati, port dilepas
- Ganti kode → **harus restart**, tidak ada auto-reload

Package "net/http"

Slide 3

- Package **standar** dari Go untuk HTTP
- Isinya:
  - **Server** — `http.ListenAndServe`
  - **Routing** — `http.ServeMux`, `mux.HandleFunc`
  - **Request & response** — `http.Request`, `http.ResponseWriter`
  - **Utilitas** — `http.Error`, `http.Redirect`, `http.NotFound`
- Contoh:

```
package main

import (
    "fmt"
    "net/http"
)
```

```
func main() {
    mux := http.NewServeMux()

    mux.HandleFunc("GET /", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprint(w, "Halo!")
    })

    http.ListenAndServe(":8080", mux)
}
```

- Perhatikan pattern-nya:
    1. `NewServeMux()` — buat router
    2. `HandleFunc(pola, fungsi)` — daftarkan pola URL
    3. `ListenAndServe(alamat, mux)` — nyalakan server

Anatomi Handler

Slide 4

- Handler = fungsi yang menerima satu request, menulis satu response

```
func haloHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Halo! Kamu mengakses %s", r.URL.Path)
}
```

- Selalu menggunakan dua parameter, **tanpa return**
  - `w http.ResponseWriter` — untuk output, tempat **menulis** jawaban
  - `r *http.Request` — data masuk, isi **permintaan** client
- Response tidak dikembalikan, tapi **ditulis** ke `w`
- Pola `return` lebih awal:

```
func produkHandler(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id")
```

```
    if id == "" {
        http.Error(w, "id wajib diisi", http.StatusBadRequest)
        return   // WAJIB. Tanpa ini, baris bawah tetap jalan
    }
    fmt.Fprintf(w, "Produk %s", id)
}
```

.

“ServeMux”: Router Bawaan Go

Slide 5

- `ServeMux` = tabel **pola URL → handler**

```
mux := http.NewServeMux()

mux.HandleFunc("GET /", homeHandler)
mux.HandleFunc("GET /about", aboutHandler)
mux.HandleFunc("GET /halo", haloHandler)

http.ListenAndServe(":8080", mux)
```

- Dua cara mendaftar:
  - `mux.HandleFunc(pola, fungsi)` — untuk fungsi biasa
  - `mux.Handle(pola, handler)` — untuk yang punya method `ServeHTTP`

Pola Route sejak Go 1.22

Slide 6

- **Dulu** (pra-1.22), `ServeMux` cuma cocokkan **awalan path**:
  - Ambil `42` dari `/produk/42`? Potong string sendiri
  - Bedakan GET/POST? `if r.Method == "POST"` di dalam handler
- **Sekarang** pola route berbentuk:

```
[METHOD ][host]/[path]
```

a) Method di depan pola

```
mux.HandleFunc("GET /produk", listProduk)
mux.HandleFunc("POST /produk", simpanProduk)
```

- Tanpa method → semua method diterima

b) `{nama}` — wildcard satu segmen

```
mux.HandleFunc("GET /produk/{id}", detailProduk)

func detailProduk(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id")        // "42" untuk /produk/42
}
```

- Satu segmen: `/produk/42` cocok, `/produk/42/review` tidak

c) `{nama...}` — sisa path, wajib di akhir

```
mux.HandleFunc("GET /files/{path...}", serveFile)
// /files/img/logo.png ? PathValue("path") == "img/logo.png"
```

d) `{$}` — hanya kalau URL berhenti di situ

```
mux.HandleFunc("GET /{$}", homeHandler)   // hanya "/"
```

- Tanpa `{$}`, pola `/` = catch-all, `/ngawur` ikut masuk

Prioritas Pola: yang Paling Spesifik Menang

Slide 7

- Urutan pendaftaran **tidak berpengaruh**

```
mux.HandleFunc("GET /produk/{id}", detailProduk)
mux.HandleFunc("GET /produk/baru", formProduk)   // didaftar belakangan
```

- `GET /produk/baru` → tetap ke `formProduk`. Literal menang atas wildcard

Tidak ada yang lebih spesifik → panic saat startup

```
mux.HandleFunc("/a/{x}", handlerA)
mux.HandleFunc("/{y}/b", handlerB)
```

```
panic: pattern "/{y}/b" conflicts with pattern "/a/{x}":
/{y}/b and /a/{x} both match some paths, like "/a/b".
But neither is more specific than the other.
```

Pola berakhiran `/`

- `/produk/` cocok dengan `/produk/` dan **semua** di bawahnya
- `/produk` (tanpa `/`) → redirect **307** ke `/produk/`

`http.ResponseWriter`: Menulis Response

Slide 8

- Response punya tiga bagian:
  - **Status code** — 200, 404, 500
  - **Header** — `Content-Type`, `Location`, `Cache-Control`
  - **Body** — isi yang dibaca browser
- Menulis body:

```
fmt.Fprint(w, "Halo, dunia!")
fmt.Fprintf(w, "Halo, %s!", nama)
```

- Header dan status:

```
w.Header().Set("Content-Type", "application/json")
w.WriteHeader(http.StatusCreated)   // 201
```

Urutan tidak boleh terbalik

```
w.Header().Set("Content-Type", "text/html; charset=utf-8")  // 1. header
w.WriteHeader(http.StatusOK)                                // 2. status
fmt.Fprint(w, "<h1>Halo</h1>")                              // 3. body
```

- Kalau terbalik, header & status **diabaikan tanpa error**:

```
fmt.Fprint(w, "body duluan")         // status terkunci 200 di sini
w.WriteHeader(http.StatusTeapot)     // diabaikan
```

- Go akan mengatur secara otomatis:
  - `WriteHeader` tidak dipanggil → 200
  - `Content-Type` tidak diset → ditebak dari 512 byte pertama body

`*http.Request`: Membaca Request

- Semua info request masuk ada di `r`:
  - `r.Method` — `"GET"`, `"POST"`
  - `r.URL.Path` — `/produk/42`
  - `r.URL.Query()` — query string, bertipe `url.Values`
  - `r.PathValue("id")` — wildcard dari pola route
  - `r.Header` — header request
  - `r.Body` — body request
  - `r.Context()` — batal otomatis kalau client putus koneksi

```
mux.HandleFunc("GET /produk/{id}", func(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id")             // dari path
    urut := r.URL.Query().Get("urut")   // dari ?urut=harga
    ua := r.Header.Get("User-Agent")    // dari header
})
```

Bedakan antara path dan query

- `r.PathValue("id")` → dari **path**: `/produk/42`
- `r.URL.Query().Get("id")` → dari **query**: `/produk?id=42`
- `Query().Get()` return `""` kalau key tidak ada — tidak error, tidak panic

```
urut := r.URL.Query().Get("urut")
if urut == "" {
    urut = "nama"     // default
}
```

- Bedakan "tidak dikirim" vs "dikirim tapi kosong":

```
if r.URL.Query().Has("urut") { ... }
```

Status Code HTTP

Slide 10

- Angka 3 digit di baris pertama response. Digit pertama menentukan arti.
- **2xx — berhasil**
  - `200 OK` — permintaan beres
  - `201 Created` — resource baru terbentuk, biasanya balasan POST
  - `204 No Content` — beres, memang tidak ada body
- **3xx — cari di tempat lain**
  - `301 Moved Permanently` — pindah selamanya, browser cache
  - `302 Found` / `303 See Other` — sementara; `303` dipakai setelah POST
  - `304 Not Modified` — pakai versi cache
- **4xx — client yang salah**

    - `400 Bad Request` — bentuk request tidak valid
    - `401 Unauthorized` — belum login
    - `403 Forbidden` — sudah login, tidak berhak
    - `404 Not Found` — resource tidak ada
    - `405 Method Not Allowed` — path ada, method salah
- **5xx — server yang salah**
    - `500 Internal Server Error` — kode kita error
    - `503 Service Unavailable` — server tidak sanggup melayani
- Pakai konstanta yang disediakan Go:

```
w.WriteHeader(http.StatusNotFound)   // bukan w.WriteHeader(404)
```

404, 405, dan Halaman Error Sendiri

Slide 11

Dengan pola Go 1.22, keduanya sudah ditangani `ServeMux` tanpa kode tambahan:

```
mux.HandleFunc("GET /{$}", homeHandler)
mux.HandleFunc("POST /produk", simpanProduk)
```

- `GET /tidak-ada` → **404 Not Found**
- `GET /produk` → **405 Method Not Allowed**, lengkap dengan header `Allow: POST`

Header `Allow` itu bagian dari spesifikasi HTTP: memberi tahu client method apa yang sebenarnya boleh dipakai.

Memanggil 404 dari dalam handler

Route-nya cocok, tapi datanya tidak ada:

```
func detailProduk(w http.ResponseWriter, r *http.Request) {
    p, ada := katalog[r.PathValue("id")]
    if !ada {
        http.NotFound(w, r)     // 404 + "404 page not found"
```

```
        return
    }
    fmt.Fprintf(w, "Produk: %s", p)
}
```

Untuk pesan sendiri, pakai `http.Error`:

```
http.Error(w, "produk tidak ditemukan", http.StatusNotFound)
```

Halaman 404 buatan sendiri

`ServeMux` tidak punya slot khusus "NotFoundHandler". Caranya adalah mendaftarkan pola paling longgar, yaitu `/` tanpa `{$}`:

```
mux.HandleFunc("/", notFoundHandler)        // menangkap semua sisanya
mux.HandleFunc("GET /{$}", homeHandler)
mux.HandleFunc("GET /about", aboutHandler)

func notFoundHandler(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusNotFound)
    fmt.Fprintf(w, "Halaman %s tidak ada. Kembali ke /", r.URL.Path)
}
```

> *Ada harganya: `/` menangkap semua method, jadi 405 otomatis ikut hilang — `GET /produk` yang tadinya 405 berubah jadi 404. Pilih salah satu sesuai kebutuhan.*

Praktek: Server dengan Beberapa Route

Slide 12

```
mkdir server-pertemuan3 && cd server-pertemuan3
go mod init server-pertemuan3
```

`main.go`:

```
package main

import (
    "fmt"
    "log"
    "net/http"
)

var katalog = map[string]string{
    "1": "Keyboard mekanik",
    "2": "Mouse wireless",
}

func homeHandler(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "text/html; charset=utf-8")
    fmt.Fprint(w, "<h1>Halaman Utama</h1><a href='/about'>Tentang</a>")
```

```
}

func aboutHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, "Server latihan Web Programming, dibuat dengan net/http.")
}

func haloHandler(w http.ResponseWriter, r *http.Request) {
    nama := r.URL.Query().Get("nama")
    if nama == "" {
        nama = "Pengunjung"
    }
    fmt.Fprintf(w, "Halo, %s!\n", nama)
}

func detailProdukHandler(w http.ResponseWriter, r *http.Request) {
    id := r.PathValue("id")
    nama, ada := katalog[id]
    if !ada {
        http.Error(w, "produk "+id+" tidak ditemukan", http.StatusNotFound)
        return
    }
    fmt.Fprintf(w, "Produk %s: %s\n", id, nama)
}

func simpanProdukHandler(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusCreated)
    fmt.Fprintln(w, "Produk tersimpan.")
}

func main() {
```

```
    mux := http.NewServeMux()

    mux.HandleFunc("GET /{$}", homeHandler)
    mux.HandleFunc("GET /about", aboutHandler)
    mux.HandleFunc("GET /halo", haloHandler)
    mux.HandleFunc("GET /produk/{id}", detailProdukHandler)
    mux.HandleFunc("POST /produk", simpanProdukHandler)

    log.Println("Server jalan di http://localhost:8080")
    log.Fatal(http.ListenAndServe(":8080", mux))
}
```

Menguji Server

Slide 13

Lewat browser (hanya bisa GET):

- `http://localhost:8080/` → halaman utama
- `http://localhost:8080/halo?nama=Andi` → `Halo, Andi!`
- `http://localhost:8080/halo` → `Halo, Pengunjung!`
- `http://localhost:8080/produk/1` → `Produk 1: Keyboard mekanik`
- `http://localhost:8080/produk/99` → 404 dengan pesan kita sendiri
- `http://localhost:8080/ngawur` → 404 bawaan, karena ada `{$}`

Untuk POST dan untuk melihat status code, pakai `curl` di terminal:

```
curl -i http://localhost:8080/produk/1     # -i menampilkan status + header
curl -i -X POST http://localhost:8080/produk
curl -i http://localhost:8080/produk       # GET ke route yang cuma menerima POST
```

Hasil yang seharusnya muncul:

```
GET  /produk/1   -> 200 OK
POST /produk     -> 201 Created
GET  /produk     -> 405 Method Not Allowed
                    Allow: POST
GET  /ngawur     -> 404 Not Found
```

Naik level: `http.Server` dengan timeout

`http.ListenAndServe` tidak punya timeout sama sekali. Satu client yang mengirim request berat bisa membuat masalah (menggantung).

Opsi yang lebih baik:

```
srv := &http.Server{
    Addr:         ":8080",
    Handler:      mux,
    ReadTimeout:  5 * time.Second,
    WriteTimeout: 10 * time.Second,
    IdleTimeout:  120 * time.Second,
}
log.Fatal(srv.ListenAndServe())
```

Untuk latihan, `http.ListenAndServe` sudah cukup.

Latihan dan Tugas

Slide 14

Latihan di kelas:

1. Tambah route `GET /waktu` yang menampilkan jam sekarang (`time.Now().Format("15:04:05")`)
2. Tambah `GET /produk` yang menampilkan seluruh isi `katalog`, sehingga `/produk` melayani GET dan POST
3. Buat `GET /redirect` yang mengarahkan ke `/about` dengan `http.Redirect(w, r, "/about", http.StatusSeeOther)`
4. Coba daftarkan dua pola yang saling konflik, lalu baca pesan panic-nya
5. Balik urutan `WriteHeader` dan `fmt.Fprint` di salah satu handler, lalu amati peringatan di terminal

Tugas individu:

- Server dengan minimal 5 route, di antaranya:
  - satu route memakai path parameter `{id}`
  - satu route memakai query string dengan nilai default
  - satu route `POST` yang membalas `201 Created`

    - halaman utama memakai `{$}`

- Submit: file `.go` + screenshot output `curl -i` untuk tiap route, dalam satu zip